ETL Architecture, Regex Text Processing, and Entity Resolution Pipelines

Created by Dr. David Gomillion with assistance from AI

Executive Summary & Core Concepts

In modern data engineering, raw enterprise data is rarely clean, structured, or immediately ready for operational analytics and artificial intelligence (AI) applications. Source payloads originating from disparate CRMs, legacy ERPs, web logs, and IoT devices frequently suffer from missing values, inconsistent formatting, embedded HTML markup, and duplicate entity references.

Without rigorous **Extract, Transform, Load (ETL)** or **Extract, Load, Transform (ELT)** pipelines, downstream applications suffer from the classic "Garbage In, Garbage Out" (GIGO) paradigm. This reading explores the architectural trade-offs between ETL and ELT, regular expression (Regex) text processing, string parsing, entity resolution (deduplication across disparate sources), and data quality validation pipelines.

Core Learning Objectives

1. **ETL vs. ELT Paradigms:** Evaluate when to perform data transformations on dedicated staging servers versus leveraging modern target database compute engines.
2. **Regex & Text Scrubbing:** Apply regular expressions in SQL and Python pipelines to sanitize, extract, and standardize unstructured and semi-structured text.
3. **Entity Resolution & Deduplication:** Understand deterministic vs. probabilistic matching algorithms (e.g., Levenshtein Distance, Jaro-Winkler, Soundex) to reconcile disparate entity records.
4. **Data Quality Frameworks:** Design automated validation pipelines with audit logging to catch and isolate non-compliant records before they contaminate core production schemas.

Part 1: ETL vs. ELT & The Data Quality Mandate

1.1 Architectural Trade-offs: ETL vs. ELT

Data integration pipelines move data from source applications into analytical repositories. The two primary paradigms—**ETL** and **ELT**—differ in where data transformations occur.

Table 1: Architectural comparison between traditional ETL and modern ELT paradigms.

Architectural Dimension	Traditional ETL (Extract, Transform, Load)	Modern ELT (Extract, Load, Transform)

Transformation Location	Dedicated intermediate transformation server	Target database engine or data warehouse
Data Flow Order	Source -> Intermediate Compute -> Target System	Source -> Target Staging -> Target Production
Raw Data Preservation	Raw source data is transformed before storage; unparsed details may be lost	Full raw source payload is preserved indefinitely in staging or raw lake storage
Compute Bottleneck	Limited by intermediate ETL server CPU/Memory	Scales with database cluster compute capabilities
Ideal Workloads	Strict compliance/privacy rules (must mask PII before loading); legacy target stores	High-volume cloud warehouses, semi-structured JSON/text dumps, AI pipelines

In modern cloud environments, **ELT** is often preferred because storage is cheap and scalable database compute engines can transform millions of records using parallel SQL queries. However, **ETL** remains essential when sensitive data (like SSNs or medical records) must be scrubbed before hitting storage, or when target engines lack advanced string processing capabilities.

1.2 The Five Dimensions of Data Quality

When building automated data cleansing pipelines, data engineers measure quality across five standardized dimensions:

1. **Completeness:** Are mandatory fields present? (e.g., Every service log must have a valid customer identifier).
2. **Accuracy:** Do values reflect real-world facts? (e.g., Battery cell temperature readings cannot be negative Kelvin).
3. **Consistency:** Is the data formatted uniformly across systems? (e.g., Phone numbers formatted as +1-555-0199 vs (555) 019-9).
4. **Uniqueness:** Are records free from unintended duplicates? (e.g., Multiple customer IDs representing the same real-world organization).
5. **Timeliness:** Is the data sufficiently up-to-date for operational decision-making?

Part 2: Regular Expressions (Regex) & Text Processing

2.1 Pattern Matching with Regular Expressions

Unstructured text fields—such as diagnostic notes, email addresses, and serial numbers—frequently contain noise or embedded metadata. **Regular Expressions (Regex)** provide a powerful, declarative pattern-matching language for string validation, extraction, and substitution.

Table 2: Essential Regular Expression syntax and common data engineering pattern matches.

Regex Operator / Sequence	Meaning / Matches	Common Data Engineering Application
^ / $	Start / End of string anchor	Validating exact string start and end conditions
\d or [0-9]	Any single digit	Extracting numeric identifiers or phone digits
\w or [a-zA-A0-9_]	Word character (alphanumeric + underscore)	Parsing sanitized usernames or SKU strings
+ / *	Quantifiers: One-or-more / Zero-or-more	Matching variable-length text sequences
<[^>]+>	HTML / XML tag pattern	Stripping raw web markup from technician notes
[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}	Email pattern	Validating customer contact input fields

2.2 SQL Regex Functions

Modern relational databases (including MariaDB and PostgreSQL) support native regular expression functions directly within standard queries:

1. Pattern Validation (REGEXP_LIKE or REGEXP)

Filters result sets based on structural string compliance:

```
-- Identify invalid email formats in staging data
SELECT raw_customer_email
FROM stg_gigavolt_erp
WHERE NOT raw_customer_email REGEXP '^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$';
```

2. Pattern Replacement (REGEXP_REPLACE)

Replaces text matching a pattern with a replacement string. This is invaluable for stripping embedded HTML tags or removing non-numeric characters:

```
-- Strip HTML tags from free-text notes
```

```
SELECT REGEXP_REPLACE(raw_technician_notes, '<[^>]+>', '') AS clean_notes
FROM stg_gigavolt_erp;
```

3. Substring Extraction (REGEXP_SUBSTR)

Extracts specific sub-patterns from complex string fields:

```
-- Extract equipment model codes (e.g., 'BATT-9000') from raw serial descriptions
SELECT REGEXP_SUBSTR(raw_equipment_serial, '[A-Z]{4}-[0-9]{4}') AS extracted_model
FROM stg_gigavolt_erp;
```

Part 3: Entity Resolution & Entity Deduplication

3.1 The Entity Resolution Challenge

In enterprise environments, the same real-world entity often exists across multiple source systems under slightly different names or identifiers. For example, a single client company might appear in three legacy tables as:

- "Gigavolt Energy Systems LLC"
- "Gigavolt Energy Systems, Inc."
- "GIGAVOLT ENERGY SYS"

Entity Resolution is the process of identifying, matching, and linking records across datasets that refer to the same real-world entity.

3.2 Matching Methodologies

Entity resolution techniques fall into two primary categories:

1. **Deterministic Matching:** Uses exact key comparisons across unique identifiers (e.g., matching records where Tax_ID = Tax_ID or Normalized_Email = Normalized_Email). While fast and reliable, deterministic matching fails when typographical errors or formatting differences exist.
2. **Probabilistic / Fuzzy Matching:** Calculates similarity scores between string attributes to estimate the probability that two records represent the same entity.

Table 3: Comparison of common string similarity algorithms used in entity resolution.

Algorithm	Operating Principle	Primary Use Case	Example Comparison

Soundex / Metaphone	Phonetic encoding (converts strings to codes based on English pronunciation)	Matching misspelled names that sound alike	Smith vs. Smythe -> Both encode to S530
Levenshtein Distance	Calculates minimum single-character edits (insertions, deletions, substitutions)	Measuring edit distance between short text fields	Gigavolt vs. Gigavllt -> Distance = 1
Jaro-Winkler Distance	Measures common characters while weighting prefix matches higher	Deduplicating person names and brand names	Mays Business vs. Mays Business School

3.3 Phonetic and Distance Matching Examples

In MariaDB, phonetic matching can be executed using the built-in SOUNDEX() function:

```
-- Find customer records that sound phonetically similar
SELECT c1.customer_name, c2.customer_name
FROM core_customers c1
JOIN core_customers c2
  ON SOUNDEX(c1.customer_name) = SOUNDEX(c2.customer_name)
 AND c1.customer_id < c2.customer_id;
```

For edit distance calculations, algorithms like **Levenshtein Distance** calculate a numerical metric reflecting similarity:

Levenshtein Distance ("GIGAVOLT", "GIGAVLLT") = 1 edit (Substitution of 'O' for 'L')

Similarity Ratio = 1 - (Edit Distance / Max Length) = 1 - (1 / 8) = 87.5%

When two records exceed a designated similarity threshold (e.g., $\ge 85\%$), the pipeline can group them under a single **Golden Master Entity Record**.

Part 4: Data Quality Validation Pipelines & Auditing

4.1 Automated Validation Pipeline Design

A robust pipeline does not simply ignore dirty data; it isolates invalid records into an **Audit Quarantine Table** while allowing clean records to proceed into production.

Table 4: Operational architecture of a quarantined data validation pipeline.

Pipeline Stage	Processing Action	Output Destination
Validation Check	Apply business logic rules (Format, Foreign Key checks, Numeric boundaries)	Pass / Fail classification
Valid Stream	Insert cleansed records into core schema	Production 3NF Core Tables
Quarantine Stream	Log raw record, failed rule name, error code, and ingestion timestamp	Quarantine Audit Table (audit_pipeline_errors)

4.2 Practical Reference Script

The following integrated script demonstrates a data cleansing pipeline: creating a quarantine table, validating records with regular expressions, routing failed records to an audit table, and resolving duplicate entities.

```
-- ============================================================================
-- ISTM 622: Integrated Laboratory Script
-- Focus: ETL Cleansing, Regex Validation, Audit Logging, and Soundex Matching
-- Engine: MariaDB / PostgreSQL
-- ============================================================================

CREATE DATABASE IF NOT EXISTS gigavolt_cleansing_sandbox;
USE gigavolt_cleansing_sandbox;

-- 1. Raw Staging Table
DROP TABLE IF EXISTS stg_raw_suppliers;
CREATE TABLE stg_raw_suppliers (
  raw_id INT AUTO_INCREMENT PRIMARY KEY,
  supplier_name VARCHAR(255),
  contact_email VARCHAR(255),
  phone_number VARCHAR(100),
  tax_identifier VARCHAR(100)
);
```

```
-- 2. Audit Quarantine Table for Bad Data
DROP TABLE IF EXISTS audit_quarantine_log;
CREATE TABLE audit_quarantine_log (
  quarantine_id INT AUTO_INCREMENT PRIMARY KEY,
  raw_id INT,
  failed_rule_name VARCHAR(100),
  invalid_value VARCHAR(255),
  logged_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. Production Clean Table
DROP TABLE IF EXISTS core_suppliers;
CREATE TABLE core_suppliers (
  supplier_id INT AUTO_INCREMENT PRIMARY KEY,
  clean_supplier_name VARCHAR(255) NOT NULL,
  clean_email VARCHAR(255) NOT NULL UNIQUE,
  phonetic_code VARCHAR(10)
);

-- 4. Insert Sample Messy Payload
INSERT INTO stg_raw_suppliers (supplier_name, contact_email, phone_number, tax_identifier) VALUES
('Gigavolt Power Systems', 'contact@gigavolt-power.com', '(555) 019-2831', 'TX-99102'),
('Gigavolt Power System', 'CONTACT@GIGAVOLT-POWER.COM', '5550192831', 'TX-99102'), -- Near duplicate
('Bad Formatting Co', 'invalid-email-address', 'N/A', 'INVALID-TAX'),          -- Bad email
('Alpha Battery Corp', 'sales@alphabatt.org', '+1-555-018-9922', 'US-88201');

-- 5. Quarantine Logging Step: Identify Invalid Emails
INSERT INTO audit_quarantine_log (raw_id, failed_rule_name, invalid_value)
SELECT
```

```
    raw_id,
    'INVALID_EMAIL_FORMAT',
    contact_email
FROM stg_raw_suppliers
WHERE NOT contact_email REGEXP '^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$';

-- 6. Load Validated & Standardized Records into Core Production
INSERT INTO core_suppliers (clean_supplier_name, clean_email, phonetic_code)
SELECT
    TRIM(supplier_name) AS clean_supplier_name,
    LOWER(TRIM(contact_email)) AS clean_email,
    SOUNDEX(supplier_name) AS phonetic_code
FROM stg_raw_suppliers
WHERE contact_email REGEXP '^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
ON DUPLICATE KEY UPDATE
    clean_supplier_name = VALUES(clean_supplier_name);

-- 7. Query Entity Resolution Candidates using Soundex Codes
SELECT
    s1.supplier_id AS primary_id,
    s1.clean_supplier_name AS primary_name,
    s2.supplier_id AS candidate_duplicate_id,
    s2.clean_supplier_name AS candidate_duplicate_name,
    s1.phonetic_code
FROM core_suppliers s1
JOIN core_suppliers s2
  ON s1.phonetic_code = s2.phonetic_code
 AND s1.supplier_id < s2.supplier_id;
```

Review & Preparation Questions

1. **Pipeline Architecture:** Under what business or security circumstances must an organization use an **ETL** architecture rather than an **ELT** architecture?
2. **Regex Processing:** What regular expression pattern would you use to match and extract a standardized serial number formatted as three uppercase letters, a dash, and four digits (e.g., BAT-4092) from an unformatted technician note field?
3. **Entity Resolution:** Why does exact string matching (WHERE name = name) fail in enterprise entity resolution, and how do phonetic algorithms like **Soundex** help bridge this gap?
4. **Data Validation:** Why is it advantageous to store rejected pipeline records in a dedicated quarantine audit table rather than simply dropping them with an IGNORE clause?

References & Further Reading

- **Sadalage, P. J., & Fowler, M. (2012).** *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley.
- **MariaDB Corporation.** "Regular Expressions Overview & Functions." *MariaDB Knowledge Base*.
- **Herzog, T. N., Scheuren, F. J., & Winkler, W. E. (2007).** *Data Quality and Record Linkage Techniques*. Springer Science & Business Media.
- **Kimball, R., & Caserta, J. (2004).** *The Data Warehouse ETL Toolkit*. Wiley.